



COMSATS University Islamabad, Lahore Campus
Department of Computer Engineering

ASSIGNMENT NO 2- FALL 2025

Course Title:	Operating System			Course Code:	CSC323	Credit Hours:	3(2,1)
Course	Engr. Ahmad Mudassir			Program	BCE		
Semester:	5 th	Batch:	FA23-BCE	Section:	A,B	Date:	8 th December 2025
Deadline	11 th December 2025			Maximum Marks:	20		
Student's Name:	Muhammad Ahmad			Reg. No.	CUI/ FA23-BCE-113	/LHR	

Important Instructions / Guidelines:

Note: Assignments must reflect each student's individual work. Any instance of plagiarism, duplication, or copying from other students or online sources will result in a zero (0) grade for all parties involved.

(Complex 2-Level Paging with Incomplete Page Tables)

A 64-bit system uses:

- Page size: 8 KB
- Page directory: 4096 entries
- Page table: 4096 entries per table
- Virtual address: 48 bits used

You have an incomplete page table dump:

PD[0x03A] -> PT @ frame 0x1F20

PD[0x07C] -> PT @ frame 0x00A0 (marked "lazy allocated")

PT[0x100] -> frame 0x0041 (present)

PT[0x205] -> frame 0x0ACD (dirty)

PT[0x3AB] -> INVALID

Given virtual address:

0x03A_205_1F4

Tasks:

- a) Break the virtual address into PD index, PT index, offset (show bit counts).
- b) Translate the virtual address step-by-step.
- c) Explain what happens if PT entry is "lazy allocated."
- d) Show what occurs if PT[0x205] has the dirty bit set but goes through eviction.
- e) Identify whether TLB must be flushed after this translation.(Justify)

Complex 2-Level paging with Incomplete page tables.

Given:

Page size : 8kB

page directory : 4096 entries

page table : 4096 entries/page

Virtual address : 4B bits

given Virtual address : $0x03A.205-1FH$

Task (a): Break The virtual address into PD-index, PT-index, offset.

As we know The virtual address for single level break into

Page #	offset
--------	--------

but for multilevel or 2-level The page no. is calculated by

PD-index + PT-index

↓
fetch PT-frame

↓
fetch page-frame

So virtual address for 2-level paging is

PD-index	PT-index	offset
----------	----------	--------

from given data

$$\rightarrow 2^{\text{offset}} = \text{page size}$$

$$2^{\text{offset}} = 8 \text{ kB}$$

$$2^{\text{offset}} = 2^{13}$$

$$\boxed{\text{offset} = 13 \text{ bits}}$$

$$\rightarrow 2^{\text{PD-index}} = \text{PD-entries}$$

$$2^{\text{PD-index}} = 4096$$

$$2^{\text{PD-index}} = 2^{12}$$

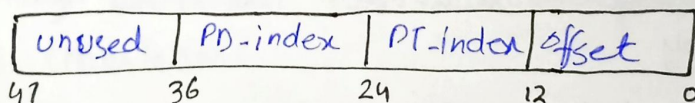
$$\boxed{\text{PD-index} = 12 \text{ bits}}$$

Similarly:

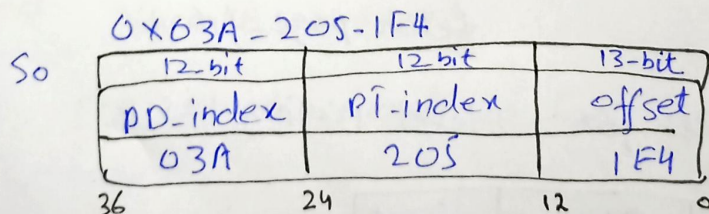
$$\boxed{\text{PI-index} = 12 \text{ bits}}$$

Address Breakdown

As we know VA is 48 bits



Address to break down



Task (b): Translate the virtual address step-by-step

(i) $PD[0x03A] \rightarrow PT @ \text{frame } 0x1F20$

↳ we first check page directory, it points to a page table at frame $0x1F20$
So page table exist and valid.

(ii) $PT[0x205] \rightarrow \text{frame } 0x0ACD$

↳ since page directory points to valid page table
so now we are checking page table, it shows frame $0x0ACD$ and it's dirty (so it's present in memory and required write back to disk on eviction)
So entry is valid but dirty.

(iii) now we just have to add offset to the PT-base index.

$PT\text{-base index} = 0ACD \ll 13$ (shift by 13 due to offset)
adding offset $0x1F4$.

So

$$\text{physical address} = (0xACD \ll 13) + 0x1F4$$

$$= (0xACD \times 2^{13}) + 0x1F4$$

$$= [(ACD \times 2000_{16}) + 1F4]_{\text{Hex}}$$

$$= [159A000 + 1F4]_{\text{hex}}$$

$$\text{Physical address} = 0x159A1F4$$

(c) Lazy Allocation Behavior of PT entry:

Lazy allocation mean OS don't allocate physical page frame until the first time the process actually try to use it. Instead of giving memory at request time OS wait until the first access (page fault).

from given dump:

~~PD~~ PD[0x07C] → "lazy allocation"

When accessed:

- System detect lazy flag.
- System allocates physical frame of PT
- update PD[0x07C] with newly allocated frame address
- Creates PT with entries marked "not present".

Benefits: Saves memory from spare address space

Cost: First access latency for allocation

First access → page fault → allocate frame → continue execution

(d) Show what occur if PT[0x205] has the dirty bit set but goes through eviction.

Dirty bit show whether the data inside page change or not

if dirty = 0, data in memory == data on disk
on eviction no write back needed

if dirty = 1, data in memory $\neq$ data on disk
on eviction write back to disk is needed

from the given dump:

PT[0x205] $\rightarrow$ frame 0x0ACD, dirty bit = 1

Eviction step:

- 1) check dirty bit: dirty bit = 1
- 2) write back: dirty bit = 1, need write back
copy 8KB page from frame[0x205]
0x0ACD and swap it on disk.
- 3) update table: reset dirty bit $\rightarrow$ to 0
- 4) free frame: 0x0ACD is free to use

If the same page is reaccessed,

page fault $\rightarrow$ load from disk $\rightarrow$ new frame allocated

(e) Identify wheather TLB must be flushed after this translation:

TLB is a hardware cache that store recently used VA $\rightarrow$ PA mapping.

No TLB flush needed for this translation because:

- This is only read only translation, no page state changes.
- no eviction occurred, mapping is still valid.